

EDA Of Large Scale Retail Transaction Data using Univariate, Bivariate and Multivariate Statistical Techniques

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Loading the Dataset
EDA = pd.read_csv("/content/retail_large_dataset.csv")
```

```
print("shape: (Columns, Rows) = ", EDA.shape)
```

shape: (Columns, Rows) = (100000, 18)

```
# The first 5 Rows
EDA.head()
```

	customer_id	age	gender	city	state	customer_segment	order_id	order_date	product_category	product_subcategory	product_price	quantity	discount_percentage	final_price
0	C10000	56	Male	Hyderabad	Telangana	New	O50000	2025-05-10	Health	Protein Powder	38658	3	7	107855.82
1	C10001	36	Male	Pune	Maharashtra	Premium	O50001	2023-11-27	Electronics	Smartphone	22462	4	20	71878.40
2	C10002	20	Male	Kolkata	West Bengal	New	O50002	2025-02-08	Fashion	Women Dress	56601	2	27	82637.46
3	C10003	39	Female	Jaipur	Rajasthan	New	O50003	2024-04-19	Beauty	Perfume	26158	3	9	71411.34
4	C10004	22	Male	Chennai	Tamil Nadu	Regular	O50004	2024-10-22	Food	Instant Noodles	21212	1	1	20000.00

```
#The last 5 Rows
EDA.tail()
```

	customer_id	age	gender	city	state	customer_segment	order_id	order_date	product_category	product_subcategory	product_price	quantity	discount_percentage	final_price
99995	C109995	29	Female	Pune	Maharashtra	Regular	O149995	2024-06-11	Health	Supplements	49293	1	11	43870.70
99996	C109996	28	Male	Mumbai	Maharashtra	Premium	O149996	2024-04-27	Fashion	Men T-Shirt	5722	2	22	8926.32
99997	C109997	44	Female	Jaipur	Rajasthan	Premium	O149997	2024-04-14	Grocery	Snacks	6311	4	11	22467.16
99998	C109998	45	Male	Jaipur	Rajasthan	Premium	O149998	2024-09-08	Beauty	Shampoo	46888	1	28	33759.36
99999	C109999	19	Male	Kolkata	West Bengal	New	O149999	2025-11-15	Health	Supplements	37361	3	3	108720.50

```
# Checking Random 30 rows
EDA.sample(30)
```

	customer_id	age	gender	city	state	customer_segment	order_id	order_date	product_category	product_subcategory	product_price	quantity	discount_percentage	final_p	
	19000	C29000	32	Female	Mumbai	Maharashtra	New	O69000	2025-06-19	Grocery	Organic Rice	44901	2	16	7543
	99177	C109177	22	Female	Delhi	Delhi	New	O149177	2024-11-22	Fashion	Women Dress	38392	1	0	3839
	99862	C109862	59	Male	Ahmedabad	Gujarat	Premium	O149862	2025-06-26	Health	Supplements	30431	1	4	2921
	26541	C36541	62	Female	Chennai	Tamil Nadu	New	O76541	2024-05-31	Health	Vitamins	55245	1	28	3977
	89794	C99794	25	Female	Ahmedabad	Gujarat	Premium	O139794	2024-06-03	Grocery	Organic Rice	45144	4	28	13001
	3470	C13470	18	Male	Kolkata	West Bengal	New	O53470	2024-12-06	Fashion	Men T-Shirt	11593	2	0	2318
	43687	C53687	22	Female	Chennai	Tamil Nadu	Regular	O93687	2023-09-13	Grocery	Snacks	20768	1	18	1702
	15738	C25738	34	Female	Mumbai	Maharashtra	Premium	O65738	2025-10-30	Fashion	Women Dress	36707	1	10	3303
	41423	C51423	39	Female	Noida	Uttar Pradesh	New	O91423	2024-03-15	Grocery	Pulses	6773	4	10	2438
	60965	C70965	36	Female	Mumbai	Maharashtra	Premium	O110965	2024-09-20	Home & Kitchen	Vacuum Cleaner	50337	1	22	3926
	10421	C20421	38	Female	Noida	Uttar Pradesh	Regular	O60421	2024-07-15	Health	Vitamins	23382	2	25	3507
	72512	C82512	60	Male	Ahmedabad	Gujarat	Regular	O122512	2023-12-27	Fashion	Jeans	38554	2	26	5705
	99243	C109243	65	Female	Kolkata	West Bengal	Premium	O149243	2025-06-04	Home & Kitchen	Cookware	59214	4	21	18711
	5637	C15637	23	Male	Chennai	Tamil Nadu	New	O55637	2025-10-08	Health	Protein Powder	2677	1	2	262
	53793	C63793	42	Female	Noida	Uttar Pradesh	New	O103793	2024-05-15	Fashion	Men T-Shirt	37514	3	29	7990
	28109	C38109	55	Female	Bangalore	Karnataka	Premium	O78109	2023-09-22	Health	Protein Powder	17963	1	16	1508
	44151	C54151	33	Male	Kolkata	West Bengal	New	O94151	2023-11-26	Beauty	Face Cream	27648	2	22	4313
	6964	C16964	49	Female	Chennai	Tamil Nadu	Regular	O56964	2024-02-10	Beauty	Shampoo	23030	1	1	2279
	46152	C56152	23	Female	Mumbai	Maharashtra	Premium	O96152	2024-08-27	Home & Kitchen	Microwave	985	3	18	242
	50489	C60489	52	Female	Bangalore	Karnataka	New	O100489	2023-07-21	Home & Kitchen	Blender	47137	4	13	16403
	71167	C81167	22	Male	Noida	Uttar Pradesh	New	O121167	2025-02-13	Electronics	Smartphone	21584	4	23	6647
	46745	C56745	59	Female	Ahmedabad	Gujarat	Premium	O96745	2024-03-31	Grocery	Snacks	30031	3	8	8288
	28229	C38229	64	Male	Ahmedabad	Gujarat	New	O78229	2024-05-17	Grocery	Snacks	23499	1	23	1809
	79522	C89522	62	Female	Jaipur	Rajasthan	New	O129522	2023-12-11	Electronics	Headphones	55376	1	6	5205
	78151	C88151	63	Female	Ahmedabad	Gujarat	Premium	O128151	2023-02-03	Electronics	Tablet	43369	3	23	10018
	59075	C69075	42	Female	Bangalore	Karnataka	Premium	O109075	2024-04-13	Home & Kitchen	Blender	21774	3	16	5487

Checking columns
EDA.columns

```
Index(['customer_id', 'age', 'gender', 'city', 'state', 'customer_segment',  
      'order_id', 'order_date', 'product_category', 'product_subcategory',  
      'product_price', 'quantity', 'discount_percentage', 'final_price',  
      'payment_method', 'shipping_type', 'delivery_days', 'return_status'],  
      dtype='object')
```

```
# Checking Datatypes  
EDA.dtypes
```

	0
customer_id	object
age	int64
gender	object
city	object
state	object
customer_segment	object
order_id	object
order_date	object
product_category	object
product_subcategory	object
product_price	int64
quantity	int64
discount_percentage	int64
final_price	float64
payment_method	object
shipping_type	object
delivery_days	int64
return_status	object

```
dtype: object
```

```
# Checking the Null values  
EDA.isnull().sum()
```

	0
customer_id	0
age	0
gender	0
city	0
state	0
customer_segment	0
order_id	0
order_date	0
product_category	0
product_subcategory	0
product_price	0
quantity	0
discount_percentage	0
final_price	0
payment_method	0
shipping_type	0
delivery_days	0
return_status	0

dtype: int64

```
# Checking Duplicates
EDA.duplicated().sum()
```

```
np.int64(0)
```

Identify the Numerical and Categorical column

```
numerical_columns = EDA.select_dtypes(include=[np.number]).columns
print(numerical_columns)
```

```
Index(['age', 'product_price', 'quantity', 'discount_percentage',
       'final_price', 'delivery_days'],
      dtype='object')
```

```
categorical_columns = EDA.select_dtypes(include=['object']).columns
print(categorical_columns)
```

```
Index(['customer_id', 'gender', 'city', 'state', 'customer_segment',
       'order_id', 'order_date', 'product_category', 'product_subcategory',
```

```
'payment_method', 'shipping_type', 'return_status'],
dtype='object')
```

Univariate Analysis

- Univariate Analysis is a type of data visualization where we visualize only a single variable at a time.

```
univariate_analysis = pd.DataFrame(EDA.describe())
univariate_analysis
```

	age	product_price	quantity	discount_percentage	final_price	delivery_days	
count	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	
mean	41.561830	30164.526720	2.501790	15.009890	64165.723382	5.505540	
std	13.852439	17139.679807	1.120058	8.953296	49969.280150	2.872868	
min	18.000000	500.000000	1.000000	0.000000	357.000000	1.000000	
25%	30.000000	15375.000000	1.000000	7.000000	24705.000000	3.000000	
50%	42.000000	30148.000000	3.000000	15.000000	49752.390000	5.000000	
75%	54.000000	44957.000000	4.000000	23.000000	94046.070000	8.000000	
max	65.000000	59998.000000	4.000000	30.000000	239980.000000	10.000000	

Next steps: [Generate code with univariate_analysis](#) [New interactive sheet](#)

```
skewness = pd.DataFrame({
    "skew": EDA[numerical_columns].skew(),
    "kurtosis": EDA[numerical_columns].kurtosis()
}).round(3)
display(skewness)
```

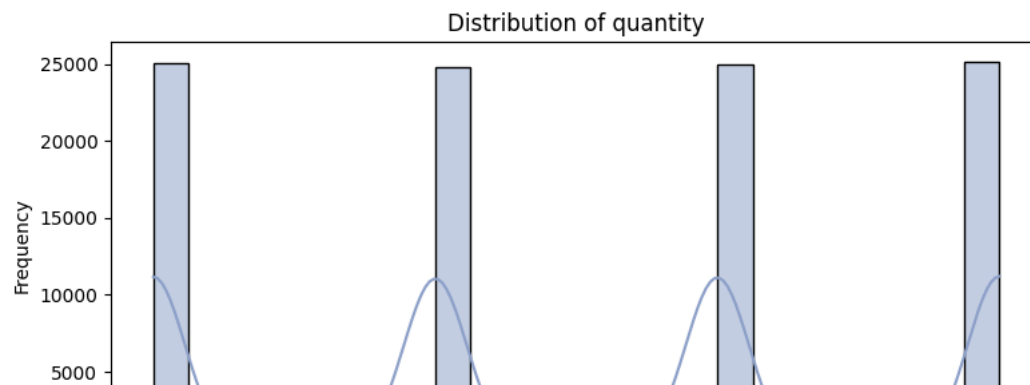
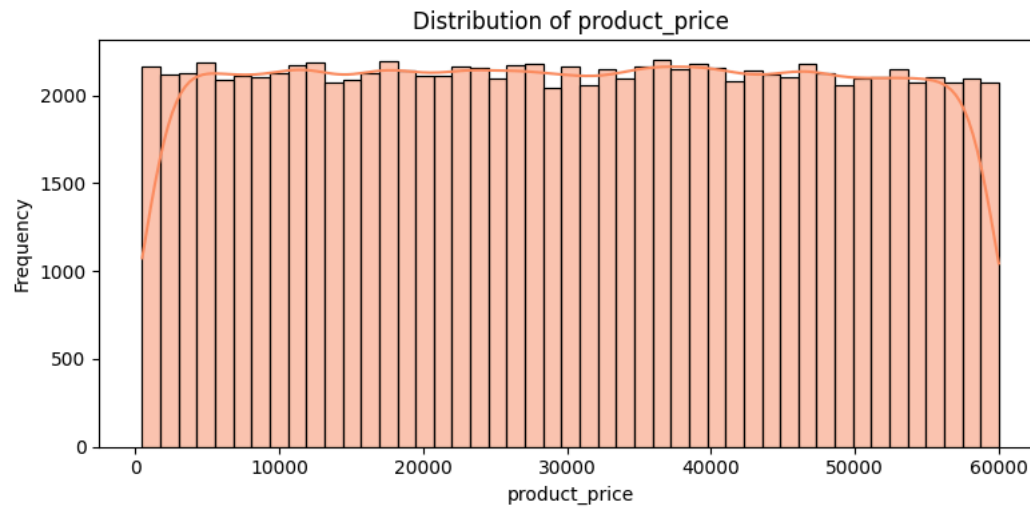
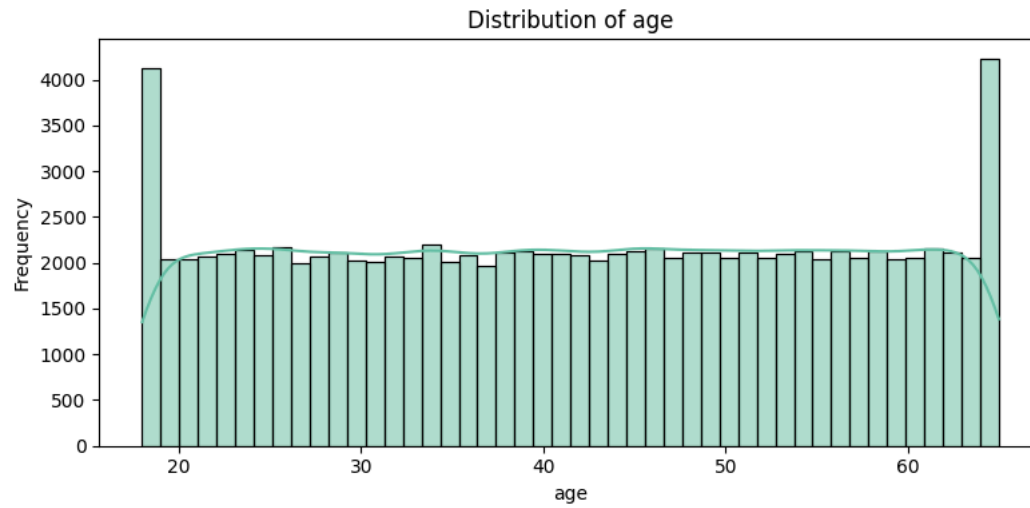
	skew	kurtosis	
age	-0.005	-1.201	
product_price	0.003	-1.195	
quantity	-0.003	-1.365	
discount_percentage	-0.004	-1.202	
final_price	0.935	0.169	
delivery_days	0.002	-1.222	

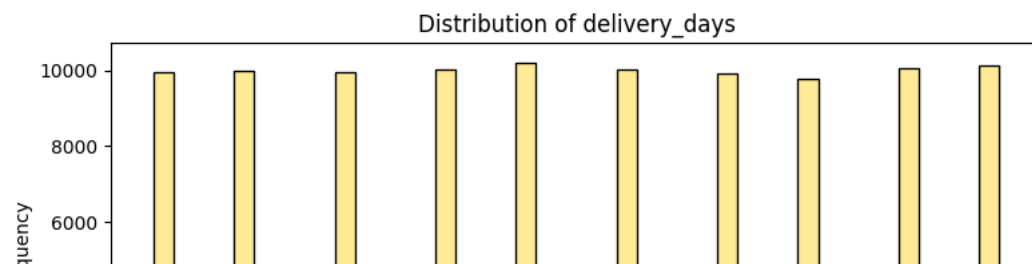
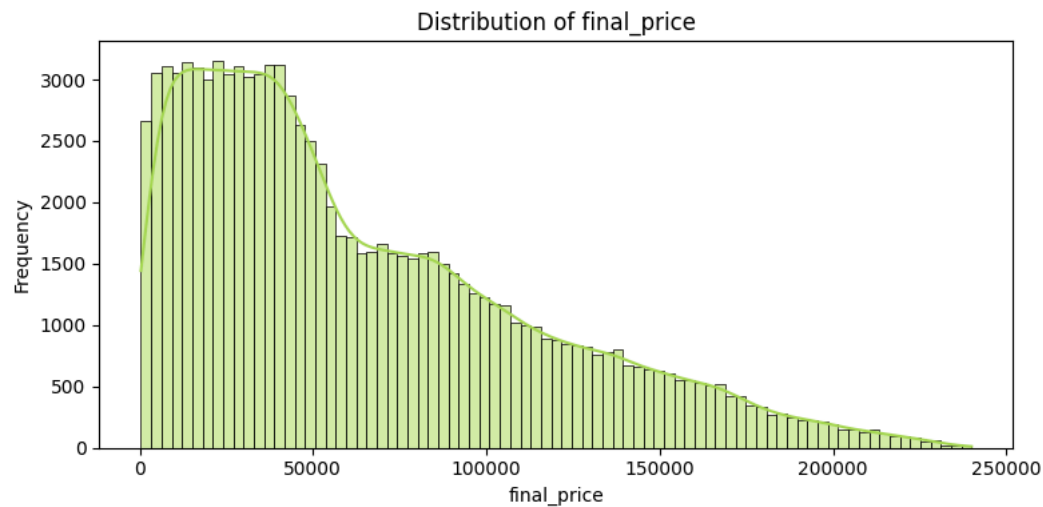
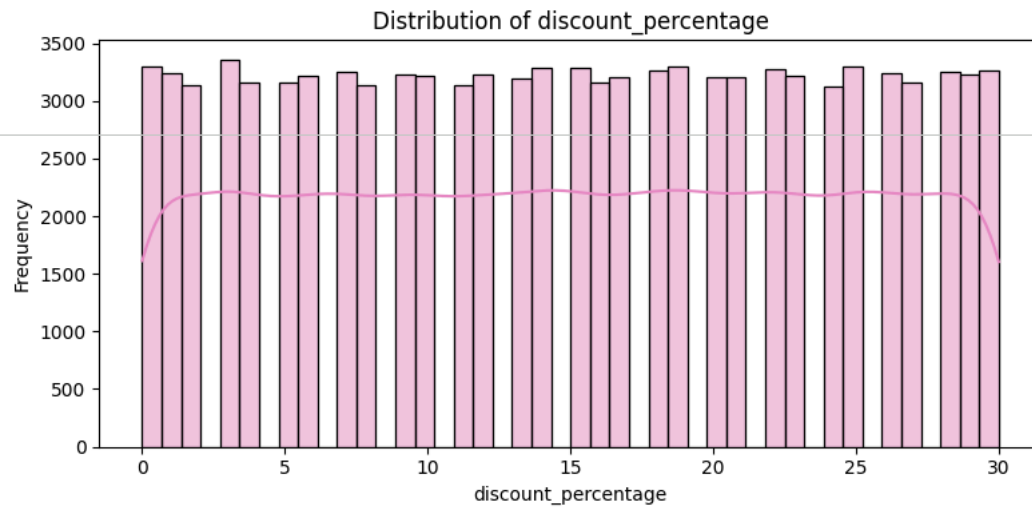
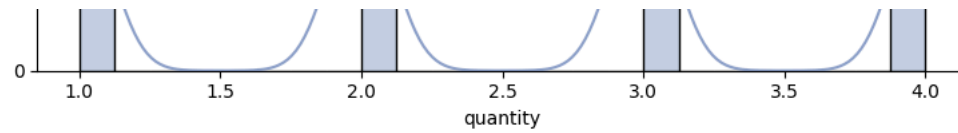
Next steps: [Generate code with skewness](#) [New interactive sheet](#)

Plotting distribution for all Numerical values

```
# Histogram for all Numerical variables
colors = sns.color_palette("Set2", len(numerical_columns))
```

```
for i, col in enumerate(numerical_columns):  
    plt.figure(figsize=(8, 4))  
    sns.histplot(EDA[col], kde=True, color=colors[i])  
    plt.title(f"Distribution of {col}")  
    plt.xlabel(col)  
    plt.ylabel("Frequency")  
    plt.tight_layout()  
    plt.show()
```



Categorical Analysis

```
categorical_columns = EDA.select_dtypes(include=['object']).columns
print(categorical_columns)
```

```
Index(['customer_id', 'gender', 'city', 'state', 'customer_segment',
       'order_id', 'order_date', 'product_category', 'product_subcategory',
       'payment_method', 'shipping_type', 'return_status',
       'delivery_days'],
      dtype='object')
```

```
categorical_var = [
    "product_category",
    "customer_segment",
    "payment_method",
    "return_status"
]

for column in categorical_var:
    print(f"\n-----{column} -----")
    display(EDA[column].value_counts())
    display((EDA[column].value_counts(normalize=True)*100).round(2))
```



-----product_category -----

count

product_category

Health	16858
Home & Kitchen	16848
Fashion	16613
Grocery	16592
Electronics	16576
Beauty	16513

dtype: int64

proportion

product_category

Health	16.86
Home & Kitchen	16.85
Fashion	16.61
Grocery	16.59
Electronics	16.58
Beauty	16.51

dtype: float64

-----customer_segment -----

count

customer_segment

New	33488
Regular	33361
Premium	33151

dtype: int64

proportion

```
for column in categorical_var:
    plt.figure(figsize=(8, 4))
    sns.countplot(data=EDA, x=column, palette="Set2")
    plt.title(f"Distribution of {column}")
    plt.xlabel(column)
    plt.ylabel("Transactions")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

-----payment_method -----

Plots for Categorical Variables

count	
payment_method	
Credit Card	25307
UPI	25139
Cash on Delivery	24994
Debit Card	24560

dtype: int64	
proportion	
payment_method	
Credit Card	25.31
UPI	25.14
Cash on Delivery	24.99
Debit Card	24.56

dtype: float64	
-----return_status -----	
count	
return_status	
No	85189
Yes	14811

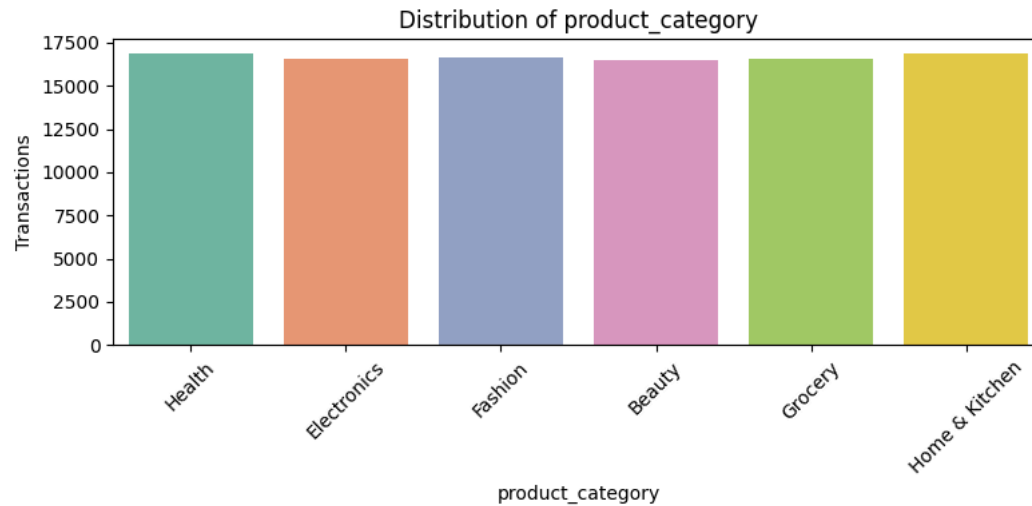
dtype: int64	
proportion	
return_status	
No	85.19
Yes	14.81

dtype: float64

```
/tmp/ipykernel_2555/2184773367.py:3: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=EDA, x=column, palette="Set2")
```



```
/tmp/ipykernel_2555/2184773367.py:3: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=EDA, x=column, palette="Set2")
```



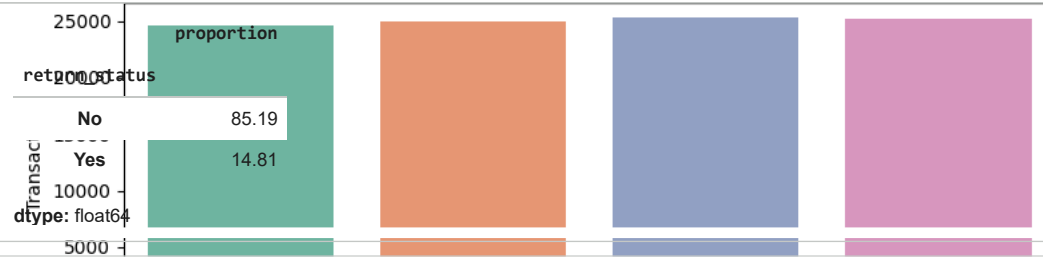
```
# Return rate = no of returned transactions / Total tranctions * 100
```

```
customer_segment
```

```
return_rate = (EDA["return_status"].value_counts(normalize=True)*100).round(2)
return_rate
```

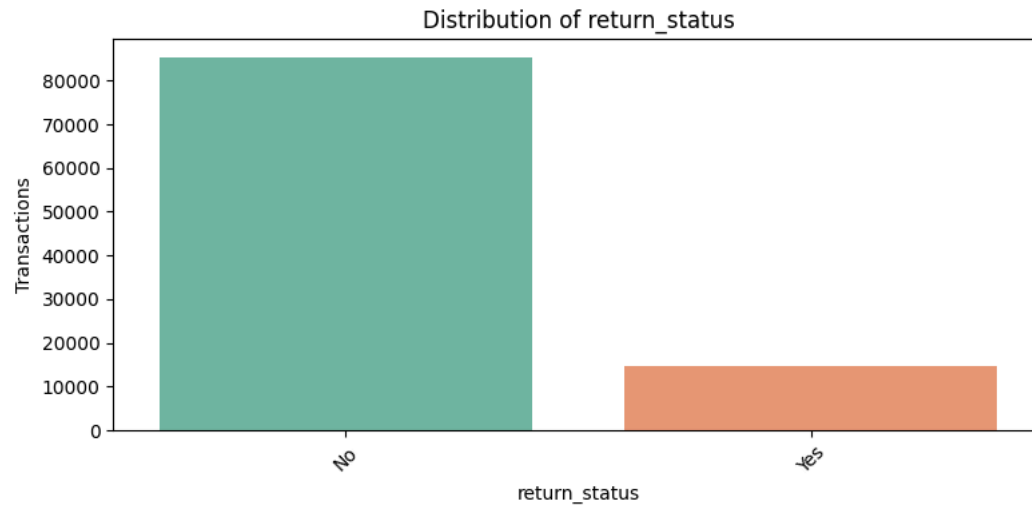
```
sns.countplot(data=EDA, x=column, palette="Set2")
```

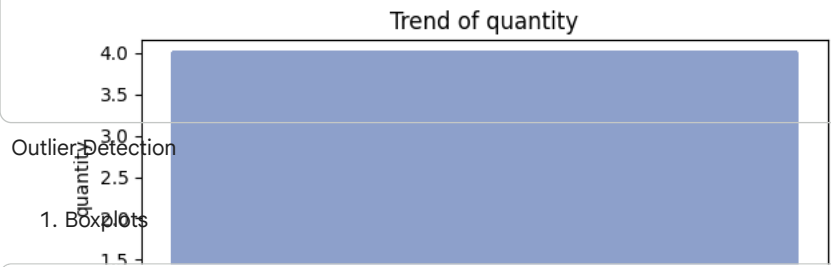
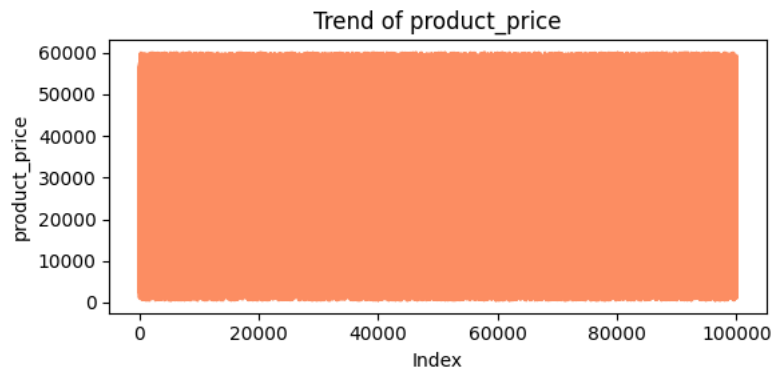
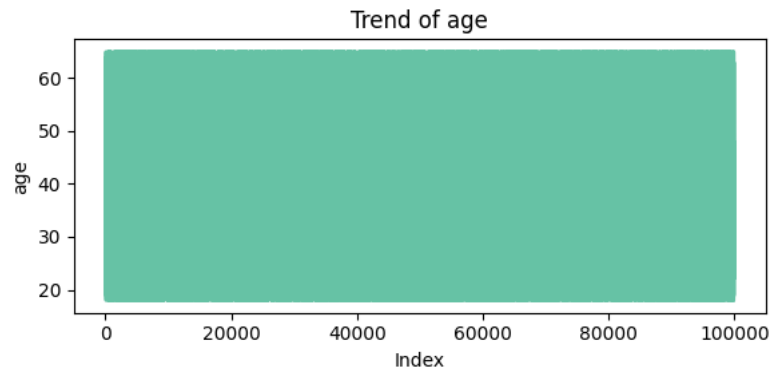
Distribution of payment_method



```
# line plot
# Line plot for all Numerical variables
colors = sns.color_palette("Set2", len(numerical_columns))

for i, col in enumerate(numerical_columns):
    plt.figure(figsize=(6, 3))
    sns.lineplot(data=EDA, x=EDA.index, y=col, color=colors[i])
    plt.title(f"Trend of {col}")
    plt.xlabel("Index")
    plt.ylabel(col)
    plt.tight_layout()
    plt.show()
sns.countplot(data=EDA, x="return_status", palette="Set2")
```

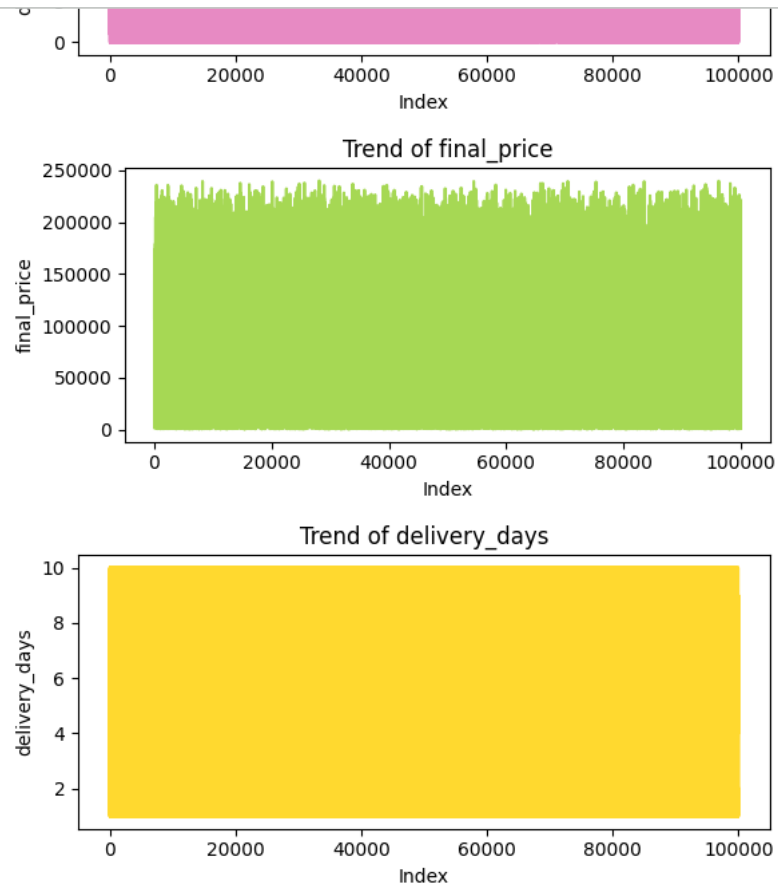




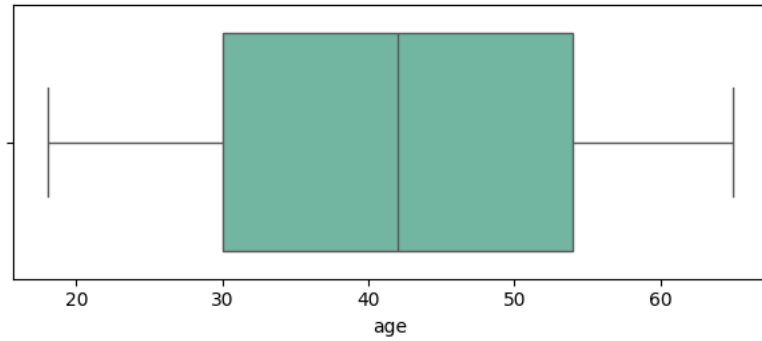
```
outlier_targets = numerical_columns
```

```
for i, column in enumerate(outlier_targets):
    plt.figure(figsize=(6,3))
    sns.boxplot(data=EDA, x=column, color=sns.color_palette("Set2")[i])
    plt.title(f"Boxplot of {column}")
    plt.tight_layout()
    plt.show()
```

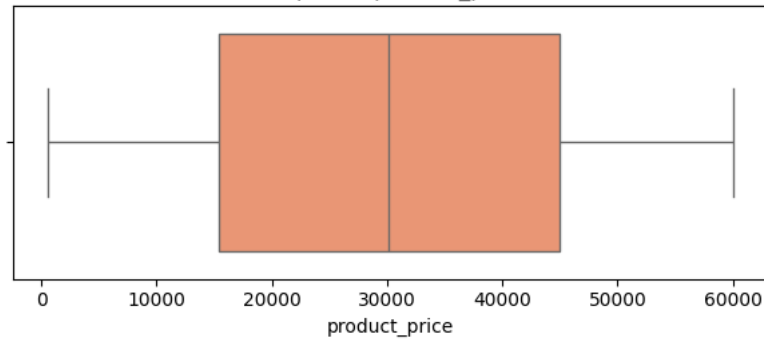




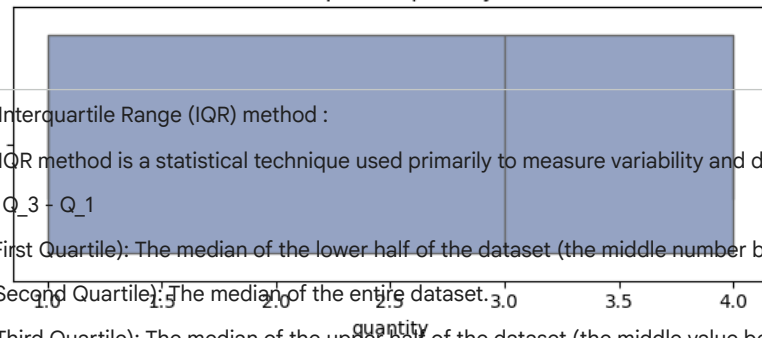
Boxplot of age



Boxplot of product_price



Boxplot of quantity



2. Interquartile Range (IQR) method :

- IQR method is a statistical technique used primarily to measure variability and detect outliers in a dataset.

$$\text{IQR} = Q_3 - Q_1$$

Q₁ (First Quartile): The median of the lower half of the dataset (the middle number between the smallest number and the median).

Q₂ (Second Quartile): The median of the entire dataset.

Q₃ (Third Quartile): The median of the upper half of the dataset (the middle value between the median and the highest value).

Boxplot of discount percentage

```
def iqr_outlier_summary(data, column):
    results = []
```

```
def iqr_outlier_summary(data, numerical_cols):
    summary_list = []
```

```
    for col in numerical_cols:
        Q1 = data[col].quantile(0.25)
```

```

Q3 = data[col].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filter outliers
outliers = data[(data[col] < lower_bound) | (data[col] > upper_bound)]
outlier_count = len(outliers)
outlier_percentage = (outlier_count / len(data)) * 100

summary_list.append({
    "Column": col,
    "Q1": Q1,
    "Q3": Q3,
    "IQR": IQR,
    "Lower Bound": lower_bound,
    "Upper Bound": upper_bound,
    "Outlier Count": outlier_count,
    "Outlier Percentage (%)": round(outlier_percentage, 2),
})

return pd.DataFrame(summary_list)

# Running the function in numerical columns
iqr_summary_df = iqr_outlier_summary(EDA, numerical_columns)
display(iqr_summary_df)

```

	Column	Q1	Q3	IQR	Lower Bound	Upper Bound	Outlier Count	Outlier Percentage (%)	
0	age	30.0	54.00	24.00	-6.000	90.000	0	0.00	
1	product_price	15375.0	44957.00	29582.00	-28998.000	89330.000	0	0.00	
2	quantity	1.0	4.00	3.00	-3.500	8.500	0	0.00	
3	discount_percentage	7.0	23.00	16.00	-17.000	47.000	0	0.00	
4	final_price	24705.0	94046.07	69341.07	-79306.605	198057.675	1335	1.33	
5	delivery_days	3.0	8.00	5.00	-4.500	15.500	0	0.00	

Next steps: [Generate code with iqr_summary_df](#) [New interactive sheet](#)

Bivariate Analysis: Examines the relationship between two variables simultaneously to determine how they influence or correlate with each other.

1. Numerical vs Numerical: Scatter Plot & Correlation To check how product_price relates to final_price:

```

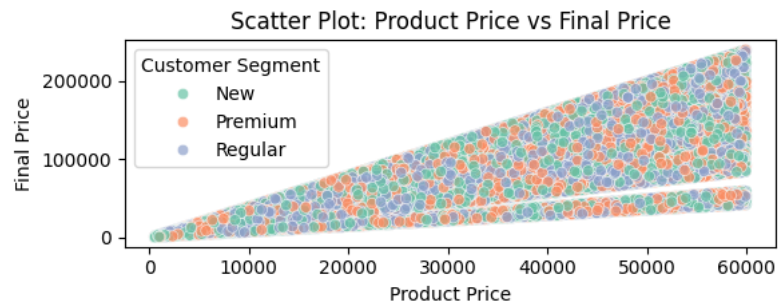
# Scatter Plot for Numerical Variables
plt.figure(figsize=(6, 2.5))
sns.scatterplot(
    data=EDA,
    x="product_price",

```

```

y="final_price",
hue="customer_segment",
palette="Set2",
alpha=0.7
)
plt.title("Scatter Plot: Product Price vs Final Price")
plt.xlabel("Product Price")
plt.ylabel("Final Price")
plt.legend(title="Customer Segment")
plt.tight_layout()
plt.show()

```



Correlation

```

# Correlation Matrix Heatmap for Numerical Variables
plt.figure(figsize=(8, 6))

# Compute correlation matrix
corr_matrix = EDA[numerical_columns].corr()

```

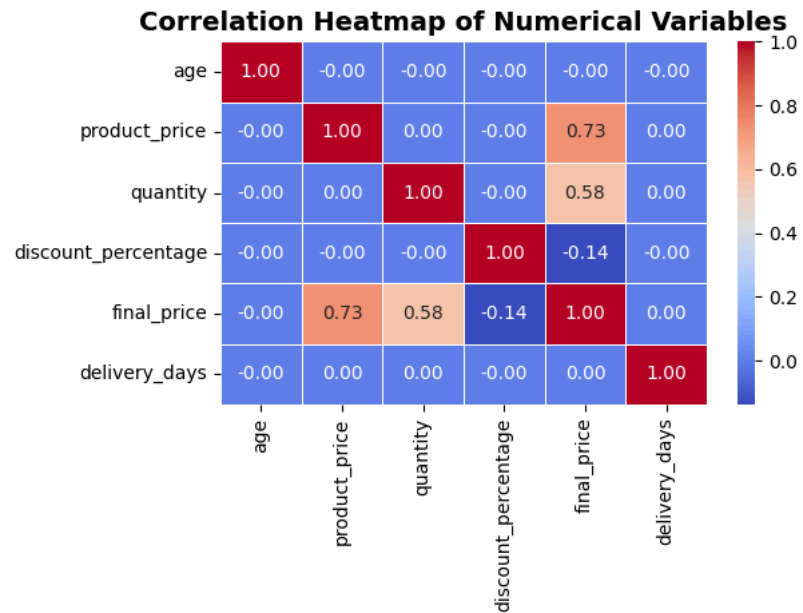
<Figure size 800x600 with 0 Axes>

```

# Draw the heatmap
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5,
    cbar=True,
)

plt.title(
    "Correlation Heatmap of Numerical Variables",
    fontsize=14,
    fontweight="bold",
)
plt.tight_layout()
plt.show()

```



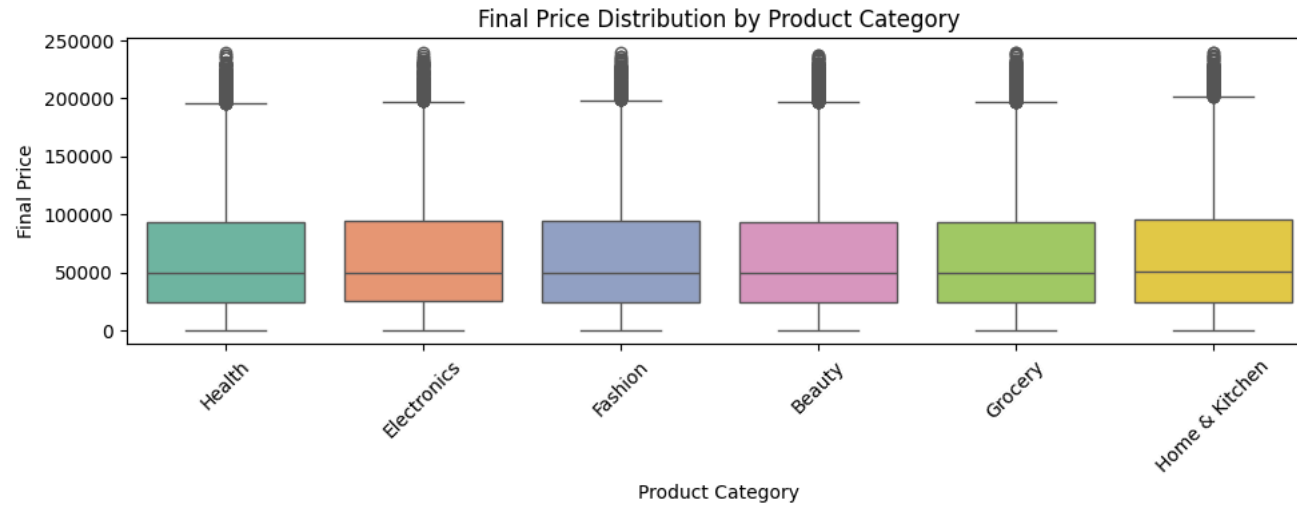
2. Categorical vs Numerical: Boxplot To compare how the final_price varies across different product_category types:

```
# Boxplot for Categorical vs Numerical Variables
plt.figure(figsize=(10, 4))
sns.boxplot(
    data=EDA, x="product_category", y="final_price", palette="Set2"
)
plt.title("Final Price Distribution by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Final Price")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_2555/1757584198.py:3: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(
```



3. Categorical vs Categorical: Grouped/Stacked Countplot To examine the relationship between customer_segment and return_status:

```
# Grouped Countplot for Categorical vs Categorical Variables
plt.figure(figsize=(8, 4))
sns.countplot(
    data=EDA, x="customer_segment", hue="return_status", palette="Set2"
)
plt.title("Return Status Across Customer Segments")
plt.xlabel("Customer Segment")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```



Multivariate analysis: Examines the simultaneous relationships among three or more variables to uncover complex patterns, interactions, and dependencies in a dataset.

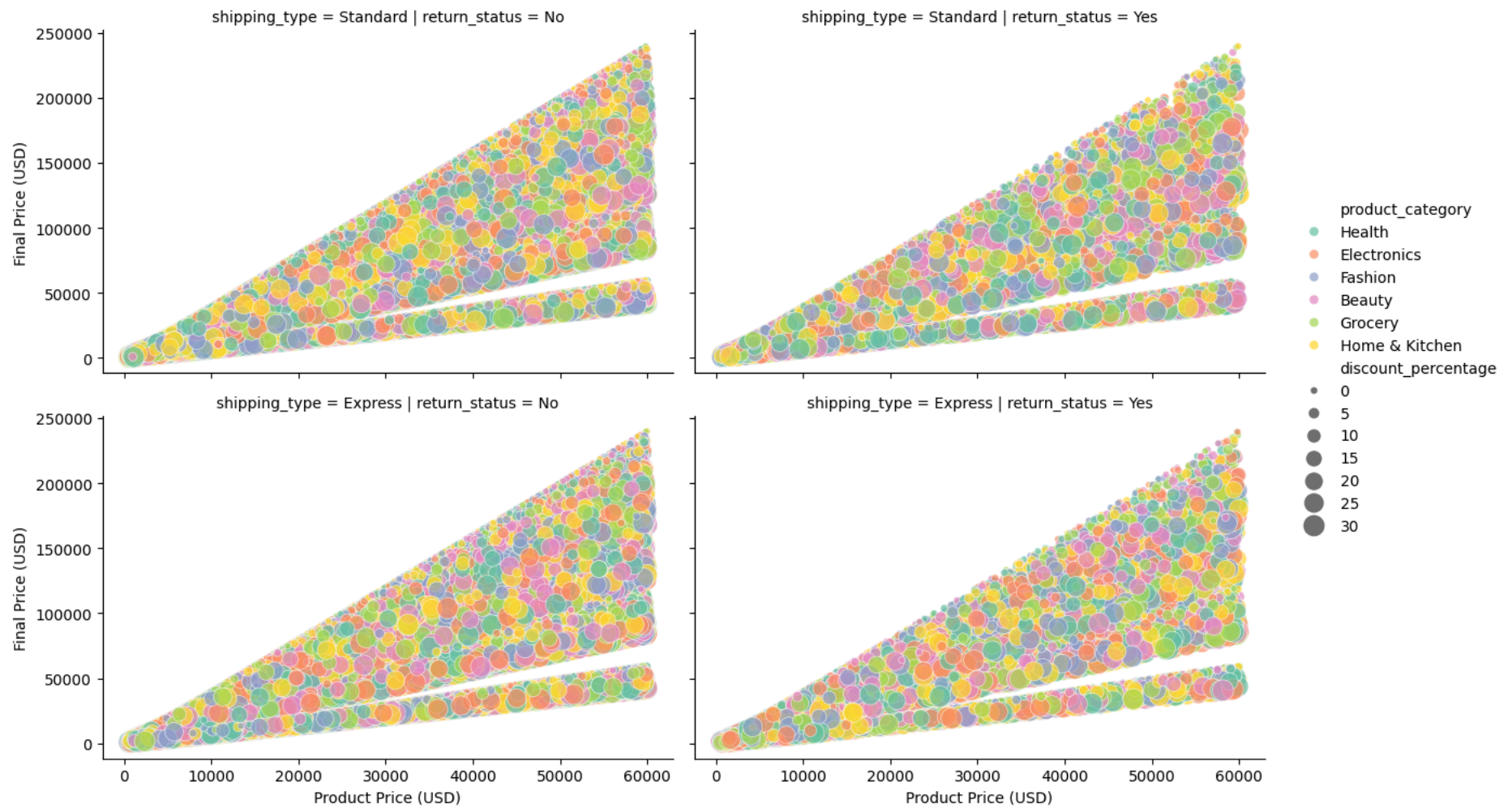
```
# Faceted or Multi-variable Scatter Plot: Discount vs Final Price grouped by Product Category
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=EDA,
    x="discount_percentage",
    y="final_price",
    hue="product_category",
    size="quantity",
    palette="Set2",
    alpha=0.7,
    sizes=(20, 150)
)
plt.title("Discount Percentage vs Final Price by Product Category & Quantity")
plt.xlabel("Discount Percentage (%)")
plt.ylabel("Final Price")
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



```
# Multi-panel multivariate bubble plot
g = sns.relplot(
    data=EDA,
    x="product_price",
    y="final_price",
    hue="product_category",
    size="discount_percentage",
    col="return_status",
    row="shipping_type",
    palette="Set2",
    alpha=0.7,
    height=4,
    aspect=1.5,
    sizes=(20, 200)
)

g.fig.suptitle("Multivariate Analysis: Product Category × Final Price", y=1.03, fontsize=14, fontweight="bold")
g.set_axis_labels("Product Price (USD)", "Final Price (USD)")
g.tight_layout()
plt.show()
```

Multivariate Analysis: Product Category × Discount × Final Price



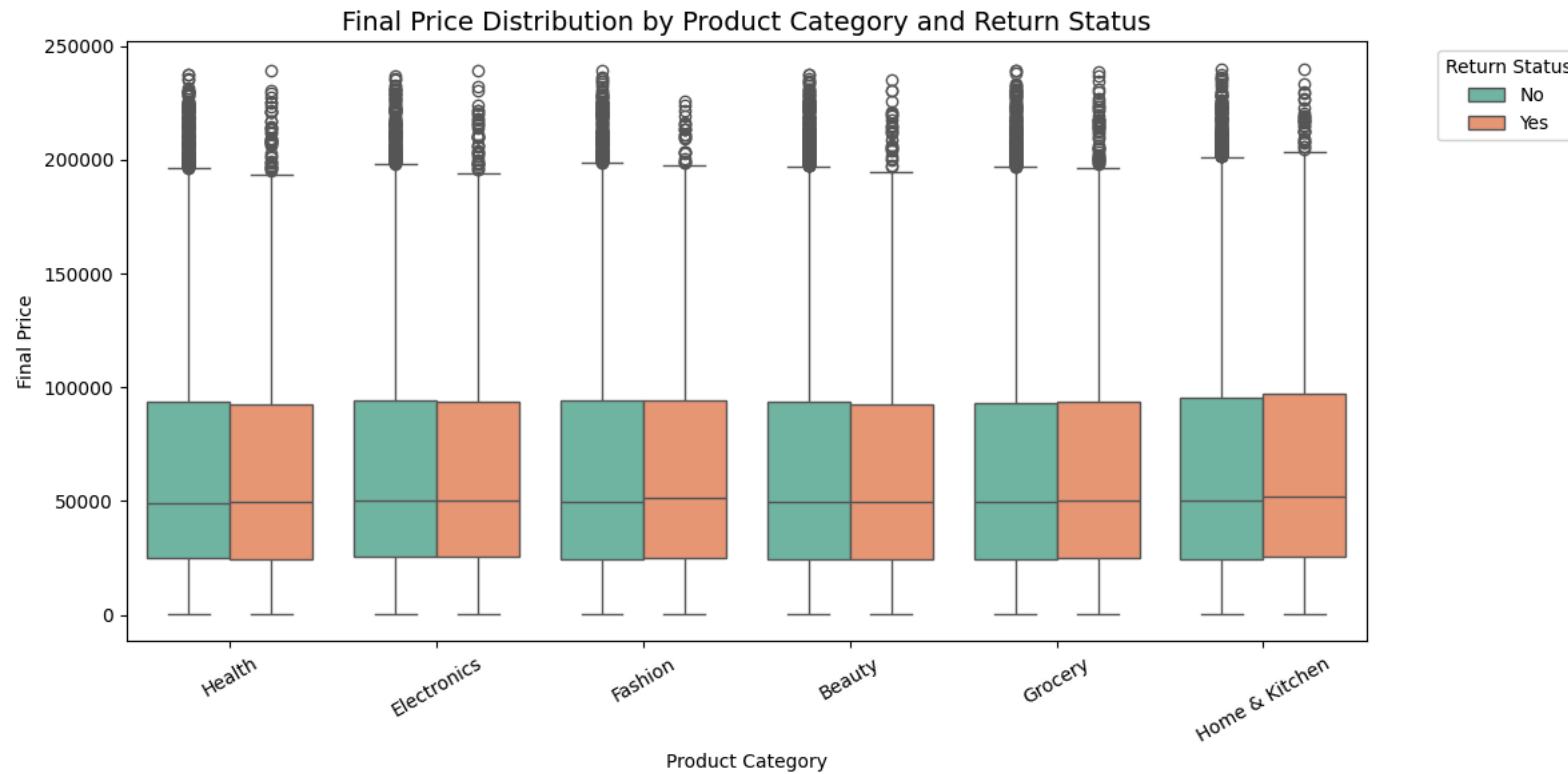
Grouped Boxplot (Categorical vs. Numerical grouped by a second Categorical variable)

```
# Grouped Boxplot to see Price Distribution across Categories and Return Status
plt.figure(figsize=(12, 6))
sns.boxplot(
    data=EDA,
    x="product_category",
    y="final_price",
    hue="return_status",
    palette="Set2",
```

```

)
plt.title(
    "Final Price Distribution by Product Category and Return Status",
    fontsize=14,
)
plt.xlabel("Product Category")
plt.ylabel("Final Price")
plt.legend(title="Return Status", bbox_to_anchor=(1.05, 1), loc="upper left")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

```



Multivariate Faceted Boxplot (Price vs Category & Customer Segment)

```

# Multivariate Analysis: Product Category vs Final Price colored by Customer Segment (or Discount/Shipping)
plt.figure(figsize=(12, 6))

# Product Category vs Final Price split/colored by Customer Segment
sns.boxplot(
    data=EDA,
    x="product_category",
    y="final_price",
    hue="customer_segment",
)

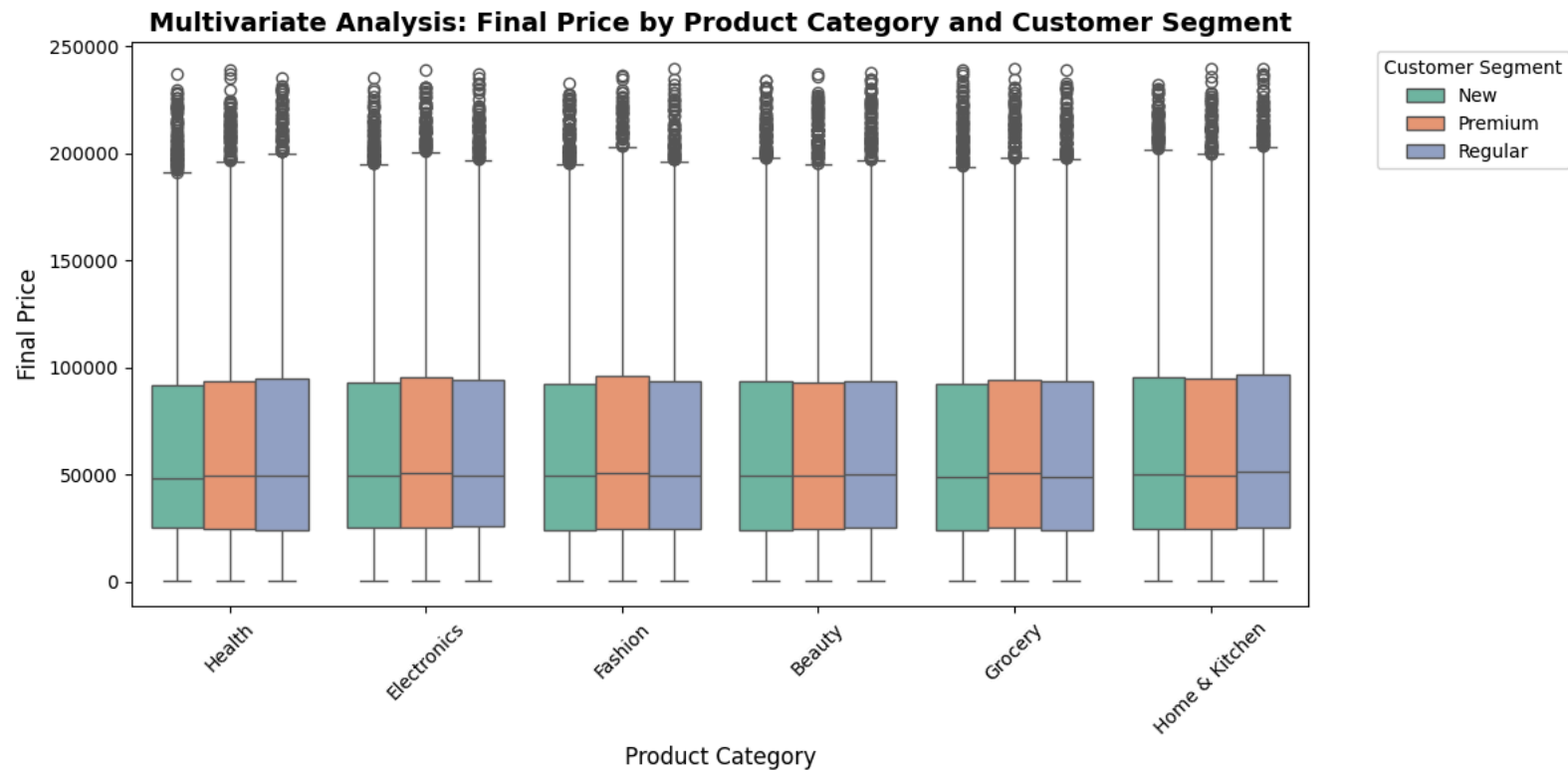
```

```

palette="Set2",
)

plt.title(
    "Multivariate Analysis: Final Price by Product Category and Customer Segment",
    fontsize=14,
    fontweight="bold",
)
plt.xlabel("Product Category", fontsize=12)
plt.ylabel("Final Price", fontsize=12)
plt.xticks(rotation=45)
plt.legend(title="Customer Segment", bbox_to_anchor=(1.05, 1), loc="upper left")
plt.tight_layout()
plt.show()

```



Seaborn Pairplot

```

# Selecting a subset of key numerical columns + a categorical column for hue
pairplot_cols = [
    "product_price",
    "quantity",
    "discount_percentage",
    "final_price",
]

```